UPDATED BUILD GUIDE
INCIDENT KNOWLEDGE BASE + CHATBOT

Dynamic, private, continuously updatable incident and solution knowledge base

Technology: Java • Spring Boot • MySQL • private document processing • keyword/full-text search • optional local vector search

Designed for the incident list and SQL-script workflow shown in your screenshots: TASK, INCIDENT, APP, DESCRIPTION, STATUS, LAST COMMENT, COMMENTS, RCA, PLAN/SOLUTION, and related SQL scripts.

Privacy rule: Never send confidential incidents, tracking keys, database details, scripts, or logs to public AI services. Keep processing inside an approved private environment.

1. What the application must do

Build a chat-based internal tool where staff search by INC number, task number, application, error text, tracking key, description, comments, RCA, or solution. The app searches only uploaded/approved records. If it finds relevant prior incidents, it shows the source records and their verified solutions. If no reliable match exists, it offers a form for the user to provide the new issue and solution. After saving and required approval, the new case becomes searchable.

Dynamic does not mean retraining an AI model for every upload. The safer design is to keep the model fixed and continuously update the stored records, versions, and search indexes.

2. Understand your incident data

- **TASK:** internal task/ticket reference.
- **INCIDENT:** incident number.
- **APP:** affected application, such as SPEI or SPID.
- **DESCRIPTION:** issue summary, possibly Spanish or English.
- **STATUS:** Pending Script, Closed, or another workflow state.
- **LAST COMMENT / COMMENTS:** investigation updates, tracking keys, operational dates, notes.
- **RCA:** root-cause analysis, when known.
- **PLAN/SOLUTION:** change request, script requirement, implementation plan, or resolution.
- **Attachments:** SQL scripts, logs, documents, and related files.

One incident may link to multiple comments, tasks, scripts, change requests, and solution versions. Preserve these relationships; do not flatten everything into one text field.

3. Payment type 16 example

Your example appears to involve payment type 16. The incident notes mention tracking keys and operation dates. The screenshots of the SQL scripts appear to select a payment record by tracking key/date, generate IDs, insert related payment/activity records, update an activity ID, and provide select/update scripts for validation or status changes. The application should store the original script as a private attachment and link it to the exact incident and approved solution.

Safety: screenshots are not enough to certify SQL as safe or correct. The chatbot must never execute retrieved SQL automatically. Store scripts for review and require an authorised operator/DBA to approve and execute them under the organisation’s change process.

4. Expected chatbot workflow

- **1 — Ask:** enter a description, INC/task number, error, or relevant context.
- **2 — Search:** check exact identifiers and structured fields, then keywords, then optional semantic/vector similarity.
- **3 — Compare:** show multiple candidate incidents, match reasons, and status; distinguish pending from resolved.
- **4 — Retrieve:** display approved RCA, comments, solution, change reference, and linked script/document metadata.
- **5 — Not found:** state that no sufficiently close approved match was found and offer “Add issue and solution.”
- **6 — Submit:** add issue details and verified solution; upload related scripts/docs.
- **7 — Review:** an authorised approver reviews the entry and marks it approved/resolved where appropriate.
- **8 — Index:** update search indexes so the approved case becomes searchable.

5. Architecture and technology

Chat UI → Spring Boot REST API → authentication/permissions → search service → MySQL
Additional private components: file storage, document/script extraction, audit trail, optional local embeddings/vector index, optional local LLM.

- **Frontend:** chat panel, candidate cards, incident details, upload form, approval queue.
- **Spring Boot:** APIs, validation, matching rules, permissions, workflows, index updates.
- **MySQL:** source of truth for incidents, comments, RCA, solutions, users, versions, and approvals.
- **Private file store:** original PDFs, spreadsheets, text files, SQL scripts, and exports.
- **Search:** exact lookup and MySQL full-text search first; optional local vector search later.
- **Optional local LLM:** summarise retrieved evidence only; not required for the first release.

Recommended stack

- Supported Java LTS compatible with chosen Spring Boot version (e.g. Java 21), Maven.
- Spring Web, Spring Data JPA, Spring Security, Bean Validation, MySQL Connector/J.
- MySQL Community Server as source of truth; Flyway for repeatable schema migrations.
- JUnit 5 and Spring Boot Test; optionally Testcontainers for MySQL integration tests.
- Local document parsing library such as Apache Tika; approved local OCR for scanned pages if required.
- Optional later: local embedding model and private vector index. Oracle is not required.

6. Suggested data model

- **incidents:** id, incident_key, task_key, app_name, title, description, status, dates, creator, timestamps.
- **incident_updates:** incident_id, update type, comment content, event time, author.
- **incident_analysis:** incident_id, RCA, plan_solution, outcome, change request, approval state.
- **incident_links:** related incidents and relation type (duplicate, similar, follow-up, same root cause).
- **tracking_references:** incident_id, tracking key, operation date, reference type; access-restrict these fields.
- **attachments:** incident_id, private storage key, filename, checksum, type, uploader, processing status.
- **script_versions:** attachment_id, version, purpose, review status, reviewer, checksum, change reference.
- **users/roles:** VIEWER, CONTRIBUTOR, APPROVER, ADMIN.
- **incident_history / audit_events:** preserve edits and record access/approval actions.

Keep source files and prior versions. Do not silently overwrite old solutions; flag conflicting resolutions for human review.

7. Import old data and compare new data

- Support approved CSV/XLSX exports from Microsoft Lists and private PDF/DOCX/TXT/SQL uploads.
- Map columns to internal fields and preview the import before saving.
- Validate required fields, dates, statuses, duplicate IDs, and missing links.
- Store original import files and an import batch ID for traceability.
- Deduplicate by stable IDs first (INC, task, script checksum); use similarity only to suggest possible duplicates.
- Create new versions instead of silently replacing existing data; flag conflicting solutions for approval.
- Show status: Uploaded → Parsed → Validated → Review (if needed) → Indexed / Failed.

8. Matching old and new incidents

- **Level 1 — exact:** INC number, task number, tracking key, error code, change reference.
- **Level 2 — structured:** app, payment type, operation date, status, transaction/reference fields.
- **Level 3 — keywords:** description, comments, RCA, solution, extracted document text.
- **Level 4 — optional vector search:** semantic similarity when wording differs.
- **Level 5 — human verification:** show candidates, match reasons, source IDs, status, and approval state.

If the match is weak, show a candidate—not a confirmed solution. If nothing passes the threshold, open the new-solution form. Never assume similar wording means identical root cause.

9. Solution submission page

When no reliable match exists, open a form that captures:

- INC/task ID if assigned; application; title and description.
- Error message, operation date, environment, and only approved identifiers.
- Investigation comments and verified RCA if known.
- Solution/plan, steps performed, outcome, validation evidence.
- Related SQL scripts, logs, and documents linked to this incident.
- Change request, reviewer, approval state, and resolution date.
- Confirmation that the submitter is authorised to upload the data.

Save as DRAFT first. After review, an authorised user marks it APPROVED/RESOLVED. A newly uploaded record must not automatically become trusted just because it exists.

10. Continuous updating and indexing

- Save new/edited records in MySQL with version history.
- Parse and validate uploads in a background job when needed.
- Update keyword/full-text indexes after approval.
- If vector search is enabled, generate embeddings locally for approved text and link them to source IDs.
- When a solution changes, preserve the old version and re-index the new version.
- When a record is archived or access is revoked, remove it from active results and update/delete vector entries.
- Show last-indexed timestamps and ingestion errors.

This is continuous knowledge-base updating—not automatic retraining. It is easier to audit and safer for sensitive incident data.

11. Optional vector search and RAG

Start without an LLM: return saved content directly. Later, use a local embedding model and private vector index for semantic matching. Optional RAG can format retrieved evidence, but must cite incident IDs/files/pages and state when evidence is insufficient.

- MySQL remains the authoritative database; vectors are a derived index.
- Generate embeddings locally or in an approved private network—no public embedding API.
- Apply app/status/date/access filters before returning results.
- Evaluate against sample questions with known correct incident IDs.
- Never treat similarity scores as probabilities; show source records.
- Never run retrieved SQL scripts automatically.

12. Security requirements

- Approved private hosting/network; block unapproved outbound connections.
- No public AI APIs or third-party file-processing services for sensitive content.
- Authentication, role-based and record-level access checks, TLS, least-privilege database user.
- Secrets outside source control; encryption for storage/backups according to policy.
- Validate uploads, store privately, use generated filenames, protect against path traversal.
- Audit uploads, edits, approvals, downloads, exports; avoid logging full confidential content.
- Retention, backup/restore, deletion, disaster recovery, index rebuild, and access review.
- Security owner approval before production data is loaded.

13. API outline

Endpoint	Purpose
POST /api/search	Search text and filters; return ranked candidates with source IDs.

GET /api/incidents/{id}	Retrieve an authorised incident.
POST /api/incidents	Create a draft issue/solution.
POST /api/incidents/{id}/attachments	Upload a private file/script linked to the incident.
POST /api/incidents/{id}/approve	Approve a version for trusted retrieval.
POST /api/imports	Start a controlled CSV/XLSX/document import.
GET /api/imports/{id}	Check parsing, validation, indexing status.
POST /api/incidents/{id}/reindex	Rebuild derived search index when authorised.

14. Build in phases

- **Phase 1:** Spring Boot skeleton, MySQL, migrations, synthetic sample records.
- **Phase 2:** incident CRUD, linked tasks/comments/RCA/solutions, history.
- **Phase 3:** exact and keyword search with source-linked results.
- **Phase 4:** authentication, roles, access checks, audit.
- **Phase 5:** solution submission, draft/approval workflow.
- **Phase 6:** private file attachments, SQL metadata, CSV/XLSX import.
- **Phase 7:** background processing, automatic re-indexing, ingestion status.
- **Phase 8:** optional local embeddings/vector search.
- **Phase 9:** optional local RAG/LLM and citation tests.
- **Phase 10:** security review, backup/restore, deployment, user acceptance tests.

15. Test cases

- Exact INC/task number returns the right incident.
- Different wording retrieves relevant candidates with source IDs.
- Similar text but different root cause is not automatically treated as solved.
- Pending Script is not represented as a completed fix.
- Missing match opens the solution submission form.
- Approved new record becomes searchable after indexing.
- Edited solution preserves history and updates search.
- Duplicate import does not silently overwrite a record.
- Unauthorised users cannot see restricted incidents or download scripts.
- No SQL script is executed by the chatbot.
- Core functions work with unapproved internet access blocked.

16. Prompt for Antigravity / coding assistant

Copy this prompt and ask the assistant to implement one phase at a time, run tests, and fix failures before continuing.

```
Build a private incident knowledge-base and chatbot using Java (supported LTS), Spring Boot, Maven, Spring Web, Spring Data JPA, Spring Security, Validation, and MySQL. Do not use PostgreSQL or Oracle. Data includes TASK, INCIDENT/INC number, APP, DESCRIPTION, STATUS, LAST COMMENT, COMMENTS, RCA, PLAN/SOLUTION, tracking keys, operation dates, change references, and linked SQL scripts/documents. Preserve relationships, source files, comments, and solution versions. Requirements: 1. Search exact identifiers/structured fields first, then keyword/full-text. Add optional local vector search only after the base system works. 2. Show candidate incidents with source IDs, match reasons, status, RCA, approved solution, and linked attachments. 3. Pending Script is not a completed fix. Never claim solved unless an approved solution exists. 4. If no reliable match exists, open a form for a user to add the issue and verified solution. 5. Save new submissions as drafts unless the user can approve. Keep history and prevent silent overwrites. 6. After approval, update indexes automatically. Show ingestion/index status and errors. 7. MySQL is the source of truth. Store files/scripts privately. Never execute SQL automatically. 8. Never send sensitive content to public AI/API services. Optional embeddings/LLM must be local or approved private deployment. 9. Implement authentication, roles, record-level permissions, safe uploads, audit, secrets management, backups, and tests in backend code. 10. Provide migrations, setup README, synthetic seed data, API docs, tests, and threat-model checklist. 11. Work incrementally: explain files changed, implement one phase, run tests, fix failures, and report honestly. Never use real production data or secrets in generated examples. Phases: project/database; incident model/history; exact and keyword search; access control; submission/approval; imports and attachments; automatic indexing; optional local vector search; optional local RAG; security/deployment.
```

17. Definition of done

- Old and new incidents import without losing IDs or source files.
- Search returns traceable records and distinguishes pending from approved/resolved.
- New solutions are submitted, reviewed, versioned, and indexed according to policy.
- Possible duplicates and conflicting solutions are flagged.
- A missing match opens the solution upload form.
- No sensitive content leaves the approved environment.
- Scripts are stored/retrieved safely but never executed automatically.
- Tests cover search, approvals, imports, permissions, uploads, and indexing.
- Deployment is approved by the organisation before production incidents are loaded.

Final recommendation

Build a dependable source-linked incident retrieval system first. Add vector search only after you have a test set of questions and expected incident IDs. The goal is not a chatbot that sounds confident; it is one that retrieves the correct approved record, explains its source, and refuses to guess when evidence is weak.